with the request and then store the token attributes as attributes of the session. Spark will take care of storing these attributes in its session database and setting the appropriate Set-Cookie header. To read token you can just check to see if a session is associated with the request, and if so, populate the `Token` object from the attributes on the session. Again, Spark takes care of checking if the request has a valid session Cookie header and looking up the attributes in its session database. If there is no valid session cookie associated with the request, then Spark will return a `null` session object, which you can then return as an `empty()` `Optional` value to indicate that no token is associated with this request.

To create the cookie-based token store, navigate to src/main/java/com/manning/apisecurityinaction/token and create a new file named CookieTokenStore.java. Type in the contents of listing 4.7 and click save.

> **WARNING** This code suffers from a vulnerability known as session fixation. You'll fix that shortly in section 4.3.1.

Listing 4.7 The cookie-based TokenStore

```
package com.manning.apisecurityinaction.token;

import java.util.Optional;
import spark.Request;

public class CookieTokenStore implements TokenStore {

    @Override
```